CUSTOM HOOKS – COMPLETE NOTES

1. What is a Custom Hook ?

- A Custom Hook is a JavaScript function whose name starts with "use" and that may call other Hooks.
- It lets you extract and reuse component logic.
- It does not work inside classes (only in functional components).

Reusability
Clean Code
Separation of Concerns

2. Why Custom Hooks ?

- ✓ Reuse logic across components.
- ✓ Keep components clean and focused on UI.
- ✓ Easier testing and maintenance.
- ✓ Share stateful logic without Redux / Zustand (when not needed).

3. Basic Syntax

```
function useSomething ( params ) {
  // use state, effect, context, etc.
  return {
    // return values / functions
  };
}
```

Must start with "use"
→
useSomething

4. Rules of Custom Hooks

- Only call Hooks at the top level.
- Do not call Hooks inside loops, conditions, or nested functions.
- Only call Hooks from React function components or other Custom Hooks.

Follow the Rules of Hooks Always
☆

5. Simple Example – useCounter

```
import { useState } from 'react'
function useCounter(initial = 0){
  const [count, setCount] = useState(initial);

  const increment = () => setCount(c => c + 1);
  const decrement = () => setCount(c => c - 1);
  const reset = () => setCount(initial);

  return { count, increment, decrement, reset };
}
```

We return values and functions to use in components.

6. Using the Custom Hook

```
function Counter() {
 const { count, increment, decrement, reset } = useCounter(10);

 return (
    <div>
       <h2>Count: {count}</h2>
       <button onClick={increment}>+1</button>
       <button onClick={decrement}>-1</button>
       <button onClick={reset}>Reset</button>
    </div>
 );
}
```

Component gets clean & focused only on UI.

★ Think of Custom Hooks as a tool box for your logic. Use it, reuse it, and keep your components simple!

☺

7. Using useEffect Inside Custom Hook

```
import { useState, useEffect } from 'react'
function useWindowSize() {
 const [size, setSize] = useState({ width: window.innerWidth,
                                    height: window.innerHeight });

 useEffect(() => {
   function handleResize() {
     setSize({ width: window.innerWidth, height: window.innerHeight });
   }
   window.addEventListener('resize', handleResize);
   return () => window.removeEventListener('resize', handleResize);
 }, []);

 return size;    // { width, height }
}
```

Custom Hook can handle side effects too ♡

8. Using useRef Inside Custom Hook - usePrevious

```
import { useRef, useEffect } from 'react'
function usePrevious(value) {
 const ref = useRef();
 useEffect(() => {
   ref.current = value;
 }, [value]);
 return ref.current;
}
```

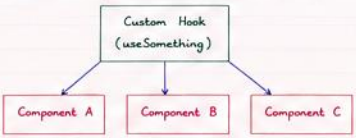
Returns previous value of something between renders.

9. Custom Hook with Input - useToggle

```
import { useState, useCallback } from 'react'
function useToggle(initial = false) {
 const [value, setValue] = useState(initial);
 const toggle = useCallback(() => setValue(v => !v), []);
 return [value, toggle];    // Array return style
}
```

Returns array (like built-in hooks).

10. Sharing Logic Between Multiple Components



One hook, many components. ☆

11. When to use Custom Hooks ?

- When same logic is used in multiple components.
- When component becomes too big and mixed with too much logic.
- When you want to separate logic (state, effect, calculations) from UI.
- When you want your code to be more maintainable and testable.

Mini Summary

- Custom Hook → Reusable logic
- Start with "use" → Rule
- Can use other Hooks → useState, useEffect, useRef, etc.
- Return values / functions → Use in components
- Keep components clean → Better structure 🎉

12. Advanced Example – useFetch

```
import { useState, useEffect } from 'react'
function useFetch(url) {
 const [data, setData] = useState(null);
 const [loading, setLoading] = useState(false);
 const [error, setError ] = useState(null);

 useEffect(() => {
  let isMounted = true;
  async function fetchData() {
    try {
      setLoading(true);
      const res = await fetch(url);
      const json = await res.json();
      if (isMounted) setData(json);
    } catch (err) {
      if (isMounted) setError(err.message);
    } finally {
      if (isMounted) setLoading(false);
    }
  }
 fetchData();
  return () => { isMounted = false }
 }, [url]);

 return { data, loading, error };
}
```

Reusable data fetching logic with loading & error states.

13. Custom Hook Best Practices

- ✓ Keep it focused – one hook, one job.
- ✓ Use meaningful names.
- ✓ Accept parameters if needed.
- ✓ Return only what is needed.
- ✓ Handle cleanup in useEffect.
- ✓ Document your hook.

Good Custom Hooks = Happy Future You ☺

14. Quick Revision

1. A Custom Hook is a function starting with "use".
2. Use other Hooks inside it.
3. Helps in reusability and clean code.
4. Can return values, functions or arrays.
5. Follow Rules of Hooks strictly.
6. Use when logic is repeated or component is messy.
7. Example Hooks → useCounter, useToggle, useFetch, usePrevious.

☆

Complete Flow

Identify Repeated Logic
↓
Create Custom Hook
↓
Use Built-in Hooks inside
↓
Return Values / Functions
↓
Use in Components
↓
Clean & Reusable Code

CUSTOM HOOKS = REUSE LOGIC + CLEAN CODE + HAPPY DEV ♥